

int y = xpx; // x = Adresse des Ints, y = Pointer-Bezeichnung
int y = xpx; // xpx = Wert einer int-Adresse, y = 5, * = Dereferenzoperator

10(16), 15(16), 256(16), 4096(16), 65536(16), 1048576(16)
BETRIEBSSYSTEM API - ZUSAMMENSPIEL OS UND PROGRAMM

Ein Programm kann aber Systemaufrufe auf Dienste des OS zugreifen: syscall → Kernel Mode des Prozessors → Argumente Programme übergeben → Argument für Nummer des Systemaufrufs OS Mechanismus um Programm zu konfigurieren: Programmargumente (explizit), Umgebungs. (implizit), Dateien oder Windows Registry (KVP).
Aufgaben: Abstraktion, Portabilität, Ressourcenmanagement & Isolation der Anwendungen, Benutzerverwaltung und Sicherheit
Privilege Levels: Kernel-Mode (darf alles ausführen, Ring 0), User-Mode (darf nur beschränkte Menge an Instructions ausführen, Ring 3)

Kernel: Mikrokernel (nur kritische Teile laufen im Kernel-Mode), **Monolithisch** (meiste OS, weniger Weicheisen, Schlüssel) (Kernel nur ein Programm)
syscall: Befehl um auf OS-Dienste zuzugreifen, wechselt in Kernel Mode von Prozessors, Argumente in Register gemäss OS (im Argument ist die Spacial Number, pro OS-Kernel-Funktion z.B. x21=60, xvec=1, C-API sollte nicht Syscall aufrufen, sondern C-Wrapper-Funktionen.
ABI: Application Binary Interface, Abstrakte Schnittstelle mit plattformunabhängigen Aspekten API: Application Programming Interface, Konkrete Schnittstellen, Calling Convention, Abbildung von Datenstrukturen. Linux-Kernels sind API-, aber nicht ABI-kompatibel, C-Wrapper-Funktionen
POSIX: Portable Operating System Interface. Sammlung von IEEE-Standards, welche die Kompatibilität zwischen OS gewährleistet. Windows ist nicht POSIX-konform.

PROGRAMMIERUNG

clang -c abc.c: Shell teilt Programmargumente in Strings auf (Trennung durch Leerzeichen, sonst Quotes). Programmarg. explizit angegeben, nützlich für Informationen, die bei jedem Aufruf anders sind.

Calling Convention

→ OS schreibt Argumente als null-terminierte Strings in den Speicherbereich des Programms.
→ Zusätzlich legt das OS ein Array arr an, dessen Elemente jeweils auf das erste Zeichen eines Arguments zeigen.
→ Der Pointer auf diese Array-Adresse und die Anzahl der Elemente wird dem Programm an einer vom OS definierten Stelle zur Verfügung gestellt z.B. in Registern oder auf dem Stack.

```
int main(int argc, char * argv[]) {  
    // 1 | argv[0] = program path  
}
```

UMGEBUNGSEINGABEN

Umgebungs- Strings, die mindestens ein Key=Value-Listen enthalten OTHER=1, PATH=home fast/Path.
Unter POSIX verwaltet das OS die Umgebungsvariablen innerhalb jedes laufenden Prozesses. Werden initial festgelegt. Das OS legt die Variablen als ein null-terminiertes Array von Pointern auf null-terminierte Strings ab.
Unter C zeigt die Variable extern char **environ darauf.

char *value = getenv("PATH"); Abfragen einer Umgebungs- returns Adresse von 1. Zeichen oder 0 wenn nicht vorhanden

```
int ret = setenv("HOME", "/usr/home", 1) / setenv(key, value, overwrite):  
Umgebungs- setzen oder über überschreiben
```

```
int ret = unset("HOME", 1) / unset(char * string): löscht vorhandenen key, string als Variable mitgeben ist im nachhinein modifizierbar (gleichlich Pointer auf NULL)
```

```
int ret = unsetenv("HOME"); Umgebungs- entfernen
```

DATENSYSTEM API

Daten: Daten (Bytes) und Metadaten (Sammlung von Attributen) Datenname, Verzeichnis, Abkürzung
Verzeichnis: Datei mit Liste aller Dateien. Verh. Hierarchie = Gesamtheit aller Dateien, im System Links: Hard-Link: gleicher Index, verschiedene Pfade, **Symbolischer Link:** wie eine Datei die Pfad auf andere Datei enthält. 0-1 Block allocated.
Zugriffsmethoden: Je 3 Prozessoren (für Owner, Gruppe und andere Benutzer.
rwx, wrx, rwx, 1 zu Zahlen so addieren, Beispiel: RWX = rwx, rwx →

Arbeitsverzeichnis: Bezugspunkt für relative Pfade, jeder Prozess hat eines (getcwd(), chdir() → nimmt String, chdir() = normal File-Descriptor). z.B. /home/bsys22

Pfade: Absolut (beginnt mit /), Relativ (beginnt nicht mit /), Kanonisch (Absolut, ohne . und .. und .., realpath())
Längster erlaubter Pfadname NAME_MAX Maximum Länge eines Datenname (exklusive terminierender Null) je nach System unterschiedlich, aber gemäss POSIX_NAME_MAX mindestens 14 chars.

Datenobjekt: Applikationen müssen Daten validieren und über Grenzwertungen überprüfen.
POSIX-API: für direkten Zugriff, alle Dateien sind roe Binärdateien. Nutzt File-Deskriptoren. Blockorientiert, list Bytes, open/close, read/write, lseek, write/read

C-API: für direkten Zugriff auf Streams, Stream-orientiert (möglicherweise gefüllte, flects/fputs (Zeilenumbruch automatisch auf OS formatiert), OS-unabhängig. fopen/fclose, fgets/fgetc/fputc/fputg, fseek/fteall

File-Descriptor: Index in die File-Descriptor-Table vom aktuellen Prozess.
File-Descriptor-Table (FDT): hat jeder Prozess, Einträge enthalten u.a. aktuellen Offset + Zeiger in global. File-Date. Nutzt Indizes in diese Tabelle als Argument für Filesystem-API-Funktionen.

Stdio, stderr: vordiensteinstreams in jedem FDT vom 0-1, wenn geschlossen können diese nicht mehr einfach geöffnet werden (im Prozess).
errno: Makro oder globale Variable vom typ int. Nach Auftreten eines Fehlers aufrufen.

FILE-DESCRIPTOR (FD)

```
#define N 32  
int main (int argc, char * argv[]) {  
    // Gibt Arbeitsverzeichnis aus  
    char *wd = malloc(PATH_MAX);  
    getcwd(wd, PATH_MAX); printf("Current WD is %s\n", wd);  
    free(wd);  
}
```

// Lesen und Schreiben von Dateien
char buf[N]; // oben definiert
char *path(PATH_MAX); // source path +
char *path(PATH_MAX); // destination path +
// ... get path from file
int src = open(path, O_RDONLY);
int dst = open(path, O_WRONLY | O_CREAT, S_IRWXO);
size_t read_bytes = read(src, buf, N);
while(dst, buf, read_bytes); // if file closed early, only write as many bytes
close(src); // as have been read from the file
close(dst);

// Datei öffnen zum Lesen und Fehlerbehandlung
int fd = open("myfile.dat", O_RDONLY);
if (fd < 0) {
 // textuelle Beschreibung von Fehlercode:
 printf("Error: %s\n", strerror(errno));
 if (errno == EACCESS) { printf("access denied"); } // eigener Text
 perror("chdir"); // (print error text to standard error stream
 // error stream new "chdir: strerror(errno)"
}

/* read data; */ close(fd);
// main

Files werden in der POSIX-API über FDs repräsentiert. Return Index in Tabelle aller geöffneten Dateien im Prozess → Enthält Index in systemweite Tabelle → Enthält Daten zur Identifikation der Datei. STDIN_FILENO = 0: standard input, STDOUT_FILENO = 1: standard output, STDERR_FILENO = 2: standard error

```
int open (char *path, int flags, ...): erzeugt FD auf Datei an path. Flags = wie die Datei geöffnet werden soll: O_RDONLY, O_WRONLY, O_CREAT(wenn nicht existiert), O_APPEND (Setzt Offset ans Ende der Datei vor jedem Schreibzugriff), O_TRUNC (Setzt Länge der Datei auf 0)
```

```
int close (int fd): Datei = deaktiviert den FD. FD kann von open oder open der Dateien verwendet werden. Returned 0 oder -1. Wenn mehrere FDs die gleiche Datei öffnen, können sie sich gegenseitig Daten überschreiben.
```

size_t read(int fd, void * buffer, size_t n): Versucht die nächsten n Bytes am aktuellen Offset von fd in den Buffer zu kopieren, erhöht FD offset, blockierend. returned: Anzahl gel. Bytes.
size_t write(int fd, void * buffer, size_t n): Versucht die nächsten n Byte vom Buffer auf einen aktuellen Offset von fd zu kopieren, erhöht FD offset, blockierend

off_t lseek(int fd, off_t offset, int origin): Setzt offset auf offset und gibt neuen Offset zurück. origin = wozu Offset relativ ist: SEEK_SET: Beginn der Datei, SEEK_CUR: Aktueller

Offset, SEEK_END: SEEKED_CUR) aktueller Offset, lseek(fd, 0, SEEK_END): Grösse der Datei. lseek(fd, n, SEEK_END) hängt, nach folgendem Vorgehen, n Nullen an Datei.
size_t pwrite(int fd, void * buffer, size_t n, off_t offset): Wie read/write. Statt Offset von fd wird zusätzlicher Parameter offset verwendet. fd offset nicht verändert.

Unterschiede Windows und POSIX

In Windows werden Pfad-Bestandteile durch Backslash (\) getrennt + es gibt ein Wurzelverzeichnis pro Dateiträger/Partition + andere File-Handling-Funktionen. (z.B. open CreateFile)

C STREAM API (FILE API)

Unabhängig vom Betriebssystem (POSIX/Windows), Stream-basiert = zeichen-orientiert, für Dateien im Normalfall gepuffert, hat einen eigenen File-Position-Indicator (FPI).

FPI: bei gepuffert Position im Puffer, bei ungepuffert Offset des FD.
Streams: FILE = <externen Funktionen (prüfen zu implementieren, aber Funktionen für, wie zip vorstellen).

Dynamische Bibliotheken: Linken zur Laufzeit bzw. Laufzeit des Programms. Executable enthält nur Referenz auf Bibliothek. Höherer Aufwand. Vorteile: austauschbar, Einkoppler Lebenszyklus, Plugin-Mechanismus. Schnellere Ladezeiten durch Lazy Loading. Flexibler Funktionsumfang).

Dynamisches Linken explizit im Programm dlopen, dlsym oder zum Programmstart.

POSIX SHARED OBJECTS API

Konventionen: Shared Objects können automatisch bei Bedarf geladen werden. Linker=Linker-Name, Loader=SO-Name.
- Linker-Name: lib + Bibliotheksname + .so (z.B. libm.so).
- SO-Name: Linker-Name + .so (z.B. libm.so).
- Real-Name: SO-Name + .so (z.B. libm.so.2.1)

Shared Objects: Nahezu alle Executables benötigen zwei Shared Objects: libc.so: Standard C library, lib-linux.so: ELF Shared Object loader (Load Shared Objects exclusively also Dependencies).

Implementierung dynamischer Bibliotheken: Müssen verschreibbar sein, müssen müssen in den gleichen Prozess geladen werden. Die Aufgabe des Linkers wird in den Loader bzw. Dynamic Linker verschoben (Load Time Relocation).

void * dlopen (char * filename, int mode): öffnet eine dynamische Bibliothek und gibt ein Handle darauf zurück. mode ist einer der folgenden Werte:
- RTLD_NOW: Alle Symbole werden beim Laden gebunden
- RTLD_LAZY: Symbole werden bei Bedarf gebunden
- RTLD_GLOBAL: Symbole können beim Binden anderer Objektdateien verwendet werden
- RTLD_LOCAL: Symbole können nicht für andere Objektdateien verwendet

void * dlsym (void * handle, char * name): gibt die Adresse des Symbols name aus der mit handle bezeichneten Bibliothek zurück. Keine Typinformationen (normaler Funktion)

// type "func_t" is an address of a function with a int param and int return type
func_t f = dlsym(handle, "func_t");
// open library
func_t f = dlsym(handle, "my_func_t"); // write my function addr. into a func_t
int i = dlsym(handle, "my_int"); // get address of "my_int"
(*f)(i); // call "my_func_t" with "my_int" as parameter

int dclose (void * handle): schliesst das handle dabei bezeichnete, zuvor geöffnete Objekt. char * dlerror(): gibt Fehlermeldung aus null-terminierten String zurück.

Position-Independent Code: (Adressen nur relativ zum Instruction Pointer, Prozessor muss relative Instructions aufrufen) Dynamische Bibliotheken sollen Code eigenen Programmen teilen. Code soll nicht mehrfach im Speicher abgelegt werden.

With Shared Memory: kann jedes Programm eine eigene virtuelle Page für den Code definieren. Diese werden auf denselben Frame im RAM gepuffert.

Relative Moves via Relative Calls: Mittels Hilfsfunktion wird Rückspundadresse in Register abgelegt, somit kann relativ dazu gearbeitet werden.

Global Offset Table (GOT): Pro dynamische Bibliothek & Executable vorhanden, enthält pro Symbol einen Eintrag. Der Loader führt zur Laufzeit die Adressen in die GOT ein.

Procedure Linkage Table (PLT): Implementiert Lazy Binding. Enthält pro Funktion einen Eintrag, dieser enthält Sprungbehl auf Adresse in GOT-Entree. Dieser zeigt auf eine Proxy-Funktion, welche den GOT-Eintrag überschreibt. Vorteil: erspart bedingten Sprung.

THREADS

Jeder Prozess hat virtuell den ganzen Rechner für sich alleine. Prozesse sind gut geeignet für unabhängige Applikationen. Nachteile: Realisierung paralleler Abläufe innerhalb derselben Applikation ist aufwändig. Overhead zu gross falls nur kürzere Teilaktivitäten, gemeinsame Ressourcennutzung ist erschwert.

Threads: parallel ablaufende Aktivitäten innerhalb eines Prozesses, welche auf alle Ressourcen im Prozess gleichermässen Zugriff haben. Benötigen eigenen Kontext und eigenen Stack. Informationen werden in einem Thread-Control-Block abgelegt.

AMDAHL'S REGEL

Nur bestimmte Teile eines Algorithmus können parallelisiert werden.
 T_p : Ausführungszeit, wenn komplett serial durchgeführt (im Bild: $T_p = T_1 + T_2 + T_3 + T_4$)
 n : Anzahl der Prozessoren
 T_p : Ausführungszeit, wenn maximal parallelisiert (gerunde Grösse)
 T_p : Ausführungszeit für den Anteil, der serial ausgeführt werden muss (im Bild: $T_p = T_1 + T_2 + T_3$)
 $T_p - T_1$: Ausführungszeit für den Anteil, der parallel ausgeführt werden kann (im Bild: $T_p - T_1$)

$(T - T_1)/n$: Parallel-Anteil verteilt auf alle n Prozessoren. (im Bild: $(T_1 + T_2)/n$)
 $T_1 + T_2/n$: Serialer Teil + Paralleler Teil = T'

Die serielle Variante benötigt also höchstens f mal mehr Zeit als die parallele Variante:

$f \leq \frac{T}{T'} = \frac{T}{T_1 + \frac{T - T_1}{n}}$

f heisst auch Speedup-Faktor, weil die parallele Variante max. f-mal schneller ist als die serielle.

Definiert man $s = T_1/T'$, also den serialen Anteil am Algorithmus, dann ist $s = T - T'$. Dadurch erhält man f unabhängig von der Zeit:

$f \leq \frac{T}{T_1 + \frac{T - T_1}{n}} = \frac{T}{s \cdot T + \frac{T - T_1}{n}} = \frac{T}{s \cdot T + \frac{T - T_1}{n}} \Rightarrow f \leq \frac{1}{s + \frac{1-n}{n}}$

Bringe s und n auf den gleichen Nenner

$f \leq \frac{1}{s + \frac{1-n}{n}} = \frac{n}{s \cdot n + 1 - n} = \frac{n}{s \cdot n + 1 - n}$

Am Ende: f Abschätzung einer oberen Schranke für den f(n) s

maximalen Geschwindigkeitsgewinn
Nur wenn alles parallelisierbar, ist Speedup proportional und maximal 100% parallel. f(n) = 1 + n - n * s

Sonst ist Speedup mit höherer Prozessoren Anzahl immer geringer (Kurve flacht ab)

Grenzwert: Mit höherer Anzahl Prozessoren nähert sich der Speedup 1 an:

$\lim_{n \rightarrow \infty} \frac{1-s}{s \cdot n + 1 - n} = 0$

$\lim_{n \rightarrow \infty} \frac{1-s}{s \cdot n + 1 - n} = s$

$\lim_{n \rightarrow \infty} \frac{1-s}{s \cdot n + 1 - n} = \frac{1}{s}$

Header: Beschreibt den Aufbau der Datei: Typ, 32/64-bit, Encoding, Maschinenarchitektur, Endianness, Infos zu den Einträgen in PHOT und SHOT.

Segment/Program Header Table und Segment: Tabelle mit n Einträgen, jeder Eintrag (je 32 Byte) beschreibt ein Segment. Typ und Flags, Offset und Grösse, virtuelle Adresse und Grösse im Speicher unterschiedlich zur physischen seq. ist Verbindung zwischen Segmenten im RAM und im File. Definiert, wo ein Segment liegt und wohin der Loader es im RAM laden soll.

Segmente werden vom Loader dynamisch zur Laufzeit verwendet.

Section Header Table und Sektionen: Tabelle mit n Einträgen (je 40 Byte)

Header: Beschreibt den Aufbau der Datei: Typ, 32/64-bit, Encoding, Maschinenarchitektur, Endianness, Infos zu den Einträgen in PHOT und SHOT.

Segment/Program Header Table und Segment: Tabelle mit n Einträgen, jeder Eintrag (je 32 Byte) beschreibt ein Segment. Typ und Flags, Offset und Grösse, virtuelle Adresse und Grösse im Speicher unterschiedlich zur physischen seq. ist Verbindung zwischen Segmenten im RAM und im File. Definiert, wo ein Segment liegt und wohin der Loader es im RAM laden soll.

Segmente werden vom Loader dynamisch zur Laufzeit verwendet.

Section Header Table und Sektionen: Tabelle mit n Einträgen (je 40 Byte)

beschreibt eine Sektion (Name, Section-Typ, Flags, Offset und Grösse...), werden vom Linker verwendet: Verschiebt Sektionen und erzeugt ausführbares Executable

String-Table: Bereich in der Datei, der nacheinander null-terminierte Strings enthält. Strings werden relativ zum Beginn der Tabelle referenziert. enthält z.B. Namen von Symbolen. Keine «String-Literale», sind in rot. in data.

Symbol- & Symboltabelle: Die Symboltabelle enthält jeweils einen Eintrag je Symbol (z.B. Byte: 48 Name, 48 Wert, 48 Grösse, 48 Info).

BIBLIOTHEKEN

Statische Bibliotheken: Archive von Objekt-Dateien. Name: lib+name>. a, referenziert wird nur <name>. Linker behandelt statische Bibliotheken wie mehrere Objekt-Dateien. Ursprünglich gab es nur statische Bibliotheken (prüfen zu implementieren, aber Funktionen für, wie zip vorstellen).

Dynamische Bibliotheken: Linken zur Laufzeit bzw. Laufzeit des Programms. Executable enthält nur Referenz auf Bibliothek. Höherer Aufwand. Vorteile: austauschbar, Einkoppler Lebenszyklus, Plugin-Mechanismus. Schnellere Ladezeiten durch Lazy Loading. Flexibler Funktionsumfang).

Dynamisches Linken explizit im Programm dlopen, dlsym oder zum Programmstart.

POSIX SHARED OBJECTS API

Konventionen: Shared Objects können automatisch bei Bedarf geladen werden. Linker=Linker-Name, Loader=SO-Name.
- Linker-Name: lib + Bibliotheksname + .so (z.B. libm.so).
- SO-Name: Linker-Name + .so (z.B. libm.so).
- Real-Name: SO-Name + .so (z.B. libm.so.2.1)

Shared Objects: Nahezu alle Executables benötigen zwei Shared Objects: libc.so: Standard C library, lib-linux.so: ELF Shared Object loader (Load Shared Objects exclusively also Dependencies).

Implementierung dynamischer Bibliotheken: Müssen verschreibbar sein, müssen müssen in den gleichen Prozess geladen werden. Die Aufgabe des Linkers wird in den Loader bzw. Dynamic Linker verschoben (Load Time Relocation).

void * dlopen (char * filename, int mode): öffnet eine dynamische Bibliothek und gibt ein Handle darauf zurück. mode ist einer der folgenden Werte:
- RTLD_NOW: Alle Symbole werden beim Laden gebunden
- RTLD_LAZY: Symbole werden bei Bedarf gebunden
- RTLD_GLOBAL: Symbole können beim Binden anderer Objektdateien verwendet werden
- RTLD_LOCAL: Symbole können nicht für andere Objektdateien verwendet

void * dlsym (void * handle, char * name): gibt die Adresse des Symbols name aus der mit handle bezeichneten Bibliothek zurück. Keine Typinformationen (normaler Funktion)

// type "func_t" is an address of a function with a int param and int return type
func_t f = dlsym(handle, "func_t");
// open library
func_t f = dlsym(handle, "my_func_t"); // write my function addr. into a func_t
int i = dlsym(handle, "my_int"); // get address of "my_int"
(*f)(i); // call "my_func_t" with "my_int" as parameter

int dclose (void * handle): schliesst das handle dabei bezeichnete, zuvor geöffnete Objekt. char * dlerror(): gibt Fehlermeldung aus null-terminierten String zurück.

Position-Independent Code: (Adressen nur relativ zum Instruction Pointer, Prozessor muss relative Instructions aufrufen) Dynamische Bibliotheken sollen Code eigenen Programmen teilen. Code soll nicht mehrfach im Speicher abgelegt werden.

With Shared Memory: kann jedes Programm eine eigene virtuelle Page für den Code definieren. Diese werden auf denselben Frame im RAM gepuffert.

Relative Moves via Relative Calls: Mittels Hilfsfunktion wird Rückspundadresse in Register abgelegt, somit kann relativ dazu gearbeitet werden.

Global Offset Table (GOT): Pro dynamische Bibliothek & Executable vorhanden, enthält pro Symbol einen Eintrag. Der Loader führt zur Laufzeit die Adressen in die GOT ein.

Procedure Linkage Table (PLT): Implementiert Lazy Binding. Enthält pro Funktion einen Eintrag, dieser enthält Sprungbehl auf Adresse in GOT-Entree. Dieser zeigt auf eine Proxy-Funktion, welche den GOT-Eintrag überschreibt. Vorteil: erspart bedingten Sprung.

THREADS

Jeder Prozess hat virtuell den ganzen Rechner für sich alleine. Prozesse sind gut geeignet für unabhängige Applikationen. Nachteile: Realisierung paralleler Abläufe innerhalb derselben Applikation ist aufwändig. Overhead zu gross falls nur kürzere Teilaktivitäten, gemeinsame Ressourcennutzung ist erschwert.

Threads: parallel ablaufende Aktivitäten innerhalb eines Prozesses, welche auf alle Ressourcen im Prozess gleichermässen Zugriff haben. Benötigen eigenen Kontext und eigenen Stack. Informationen werden in einem Thread-Control-Block abgelegt.

AMDAHL'S REGEL

Nur bestimmte Teile eines Algorithmus können parallelisiert werden.
 T_p : Ausführungszeit, wenn komplett serial durchgeführt (im Bild: $T_p = T_1 + T_2 + T_3 + T_4$)
 n : Anzahl der Prozessoren
 T_p : Ausführungszeit, wenn maximal parallelisiert (gerunde Grösse)
 T_p : Ausführungszeit für den Anteil, der serial ausgeführt werden muss (im Bild: $T_p = T_1 + T_2 + T_3$)
 $T_p - T_1$: Ausführungszeit für den Anteil, der parallel ausgeführt werden kann (im Bild: $T_p - T_1$)

$(T - T_1)/n$: Parallel-Anteil verteilt auf alle n Prozessoren. (im Bild: $(T_1 + T_2)/n$)
 $T_1 + T_2/n$: Serialer Teil + Paralleler Teil = T'

Die serielle Variante benötigt also höchstens f mal mehr Zeit als die parallele Variante:

$f \leq \frac{T}{T'} = \frac{T}{T_1 + \frac{T - T_1}{n}}$

f heisst auch Speedup-Faktor, weil die parallele Variante max. f-mal schneller ist als die serielle.

Definiert man $s = T_1/T'$, also den serialen Anteil am Algorithmus, dann ist $s = T - T'$. Dadurch erhält man f unabhängig von der Zeit:

$f \leq \frac{T}{T_1 + \frac{T - T_1}{n}} = \frac{T}{s \cdot T + \frac{T - T_1}{n}} = \frac{T}{s \cdot T + \frac{T - T_1}{n}} \Rightarrow f \leq \frac{1}{s + \frac{1-n}{n}}$

Bringe s und n auf den gleichen Nenner

$f \leq \frac{1}{s + \frac{1-n}{n}} = \frac{n}{s \cdot n + 1 - n} = \frac{n}{s \cdot n + 1 - n}$

Am Ende: f Abschätzung einer oberen Schranke für den f(n) s

maximalen Geschwindigkeitsgewinn
Nur wenn alles parallelisierbar, ist Speedup proportional und maximal 100% parallel. f(n) = 1 + n - n * s

Sonst ist Speedup mit höherer Prozessoren Anzahl immer geringer (Kurve flacht ab)

Grenzwert: Mit höherer Anzahl Prozessoren nähert sich der Speedup 1 an:

$\lim_{n \rightarrow \infty} \frac{1-s}{s \cdot n + 1 - n} = 0$

$\lim_{n \rightarrow \infty} \frac{1-s}{s \cdot n + 1 - n} = s$

$\lim_{n \rightarrow \infty} \frac{1-s}{s \cdot n + 1 - n} = \frac{1}{s}$

Header: Beschreibt den Aufbau der Datei: Typ, 32/64-bit, Encoding, Maschinenarchitektur, Endianness, Infos zu den Einträgen in PHOT und SHOT.

Segment/Program Header Table und Segment: Tabelle mit n Einträgen, jeder Eintrag (je 32 Byte) beschreibt ein Segment. Typ und Flags, Offset und Grösse, virtuelle Adresse und Grösse im Speicher unterschiedlich zur physischen seq. ist Verbindung zwischen Segmenten im RAM und im File. Definiert, wo ein Segment liegt und wohin der Loader es im RAM laden soll.

Segmente werden vom Loader dynamisch zur Laufzeit verwendet.

Section Header Table und Sektionen: Tabelle mit n Einträgen (je 40 Byte)

Header: Beschreibt den Aufbau der Datei: Typ, 32/64-bit, Encoding, Maschinenarchitektur, Endianness, Infos zu den Einträgen in PHOT und SHOT.

Segment/Program Header Table und Segment: Tabelle mit n Einträgen, jeder Eintrag (je 32 Byte) beschreibt ein Segment. Typ und Flags, Offset und Grösse, virtuelle Adresse und Grösse im Speicher unterschiedlich zur physischen seq. ist Verbindung zwischen Segmenten im RAM und im File. Definiert, wo ein Segment liegt und wohin der Loader es im RAM laden soll.

Segmente werden vom Loader dynamisch zur Laufzeit verwendet.

Section Header Table und Sektionen: Tabelle mit n Einträgen (je 40 Byte)

POSIX THREAD API

```
pthread_create(  
    pthread_t * thread_id, pthread_attr_t const * attributes,  
    void * (*start_function) (void *), void * argument)
```

erzeugt einen Thread, die ID des neuen Threads wird im Out-Parameter thread_id zurückgegeben. attributes ist ein options Objekt, mit dem z.B. die Stack-Grösse spezifiziert werden kann. Die erste auszuführende Instruktion ist die Funktion in start_function, argument ist ein Pointer auf eine Datenstruktur auf dem Heap für die Argumente für start_function.

// Erzeugung
struct t { // params of function
 int value;
};
void * start_func(void * arg) {
 struct t * p = arg;
 printf("val=%d", p->value);
 return 0;
}

// Verwendung
void start_my_thread(void) {
 struct t * t = malloc(sizeof(struct t));
 t->value = 109; // set argument
 pthread_t tid;
 pthread_create(&tid,
 0, // default attributes
 start_func, t);
}

Thread-Attribute:
pthread_attr_t attr; // Variabel erstellen
pthread_attr_t (attr); // Variabel initialisieren
pthread_attr_t set_schedulers (attr, 1 <= 10); // 640k Stackgröße
pthread_create (..., attr, ...); // Thread erstellen
pthread_attr_t destroy (attr); // Attribute löschen

Lebensdauer: Ist solange, bis er aus dem start_function zurückspringt, er pthread_exit oder ein anderer Thread pthread_cancel aufruft oder sein Prozess beendet wird.

void pthread_exit (void *return_value): Beendet den Thread und gibt den return_value zurück. Das ist äquivalent zum Rückprung aus start_function mit dem Rückgabewert.

int pthread_cancel (pthread_t tid): Sendet eine Anforderung, dass der Thread mit tid beendet werden soll. Die Funktion wartet nicht, dass der Thread tatsächlich beendet wurde. Der Rückgabewert ist 0, wenn der Thread existiert, bzw. ESRCH (error_search), wenn nicht.

int pthread_detach (pthread_t tid): Entfernt den Speicher, den ein Thread belegt hat, falls dieser bereits beendet wurde. Beendet den Thread aber nicht. (erstellt Daemon Thread)

int pthread_join (pthread_t thread_id, void *return_value): Wartet solange, bis der Thread mit tid beendet wurde. Nimmt den Rückgabewert des Threads im Out-Parameter return_value entgegen. Dieser kann NULL sein, wenn nicht gewünscht. Ruft pthread_detach auf.

pthread_t pthread_self (void): Gibt die ID des gerade laufenden Threads zurück.

THREAD-LOCAL STORAGE (TLS)

TLS ist ein Mechanismus, der globale Variablen pro Thread zur Verfügung stellt. Dies benötigt mehrere explizite Einzeichnungen: Bevor Threads erzeugt werden: Anlegen eines Keys, der die TLS-Variable identifiziert, Speichern des Keys in einer globalen Variable im Thread: Auslesen des Keys aus der globalen Variable, Auslesen / Schreiben des Werts anhand des Keys.

int pthread_key_create (pthread_key_t *key, void (*destructor) (void*)): Erzeugt einen neuen Key im Out-Parameter key. Opake Datenstruktur. Am Thread-Ende Call auf destructor.

int pthread_key_delete (pthread_key_t key): Entfernt den Key und die entsprechenden Values aus allen Threads. Der Key darf nach diesem Aufruf nicht mehr verwendet werden. Sollte erst aufrufen werden, wenn alle dazugehörigen Threads beendet sind.

int pthread_getspecific (pthread_key_t key, const void *value):
void * pthread_getspecific (pthread_key_t key) schreibt bzw. liest den Wert, der mit dem Key in diesem Thread assoziiert ist. Oft als Pointer auf einen Speicherbereich verwendet.

// Setup
typedef struct {
 int code;
 char *message;
} error_t;
pthread_key_t error;
void set_up_error (void) {
 // am Anfang des Threads aufrufen
 error_t e = {
 error, malloc(sizeof(error_t))
 };
 e->code = 98;
 e->message = "file not found";
 return e;
}

// Lesen und Schreiben in Thread
void print_error (void) {
 error_t e = pthread_getspecific(error);
 printf("Error %d: %s\n", e->code, e->message);
}

int pthread_key_create (pthread_key_t *key, void (*destructor) (void

